Progetto - Architetture Dati

Ottimizzazione di un sistema RAG

Andrea Mauri : 945327
Luca Rossini: 900230
Francesco Sormani: 894552

Anno Accademico 2025-2026

Link Repository
`https://github.com/rossini-luca/progetto-architetture-dati`

Indice

1 Introduzione **2**

2 Cenni Teorici **3**
2.1 Architettura RAG . . . 3
2.2 Chunking (Suddivisione del Testo) . . . 3
2.3 Modelli di Embedding . . . 4
2.4 Query Transformation . . . 4
2.5 Metodi di Retrieval . . . 5
2.6 LLM e Parametri di Generazione . . . 6

3 Struttura del Progetto **7**
3.1 Struttura dei File . . . 7
3.2 Librerie e Framework Utilizzati . . . 8
3.3 Scelte Architetturali e Hardware . . . 9

4 Workflow del Progetto **10**
4.1 La Fase di Ottimizzazione . . . 10
4.2 La Fase Generativa . . . 11
4.3 Metriche di Valutazione . . . 12
4.3.1 Metriche della Fase di Ottimizzazione . . . 12
4.3.2 Metriche della Fase Generativa . . . 13

5 Domini Analizzati **14**
5.1 Composizione e Scelta dei Domini . . . 14
5.2 Struttura delle Domande . . . 14

6 Analisi dei Risultati **16**
6.1 Ottimizzazione: Blocco 1 (Chunking ed Embedding) . . . 16
6.2 Ottimizzazione: Blocco 2 (Query Transformation) . . . 18
6.3 Ottimizzazione: Blocco 3 (Retrieval e Reranking) . . . 20
6.4 Fase Generativa . . . 22

7 Conclusioni **25**

Capitolo 1

Introduzione

L'obiettivo di questo progetto è la realizzazione e ottimizzazione di un sistema RAG (Retrieval-Augmented Generation). I classici LLM presentano infatti notevoli limiti nel recupero di informazioni dominio-specifiche, non avendo accesso a dati privati o successivi al loro addestramento. Per questo motivo sono soggetti al fenomeno delle allucinazioni, tendendo a inventare risposte plausibili ma fattualmente errate.

L'architettura RAG risolve questi problemi scorporando il processo in due step: prima ricerca all'interno di una base di documenti i frammenti di testo più rilevanti per la domanda dell'utente, e successivamente fornisce questi frammenti come contesto all'LLM, permettendogli di generare una risposta basata esclusivamente su dati reali.

Nel corso del progetto abbiamo creato un sistema RAG da zero, testandone e misurandone l'efficacia su due domini di testo diversi tra loro: uno tecnico (documenti RFC) e uno più narrativo (documenti sulla Prima Guerra Mondiale).

Abbiamo affrontato lo sviluppo suddividendolo in due macro-fasi:

- **Fase di Ottimizzazione**: in cui abbiamo esplorato svariate configurazioni, testando metodi di chunking, modelli di embedding, tecniche di trasformazione della query e algoritmi di reranking, allo scopo di individuare la configurazione più precisa e veloce.
- **Fase Generativa**: in cui abbiamo collegato il RAG ottimizzato a diversi modelli LLM open-source locali. Abbiamo testato varie configurazioni sperimentando diverse tecniche di *Prompt Engineering* per rispondere a differenti tipologie di domande, valutando infine la qualità del testo prodotto attraverso il metodo automatizzato *LLM-as-a-Judge*.

Nei capitoli seguenti introdurremo brevemente la teoria alla base dei componenti utilizzati, esploreremo le scelte architetturali del codice e analizzeremo passo dopo passo i risultati pratici ottenuti durante i test.

Capitolo 2

Cenni Teorici

2.1 Architettura RAG

L'architettura RAG opera combinando un sistema di Information Retrieval con le capacità di generazione testuale di un LLM. L'intero processo si struttura in due fasi principali:

- **Indicizzazione**: in questa fase i documenti originali vengono frammentati in blocchi più piccoli definiti "chunk", i quali vengono successivamente convertiti in rappresentazioni numeriche e archiviati all'interno di un database vettoriale
- **Interrogazione**: quando l'utente pone una domanda, anch'essa viene trasformata in un vettore. Il RAG estrae dal database i chunk semanticamente più vicini alla domanda e li fornisce come contesto aggiuntivo all'LLM. In questo modo il modello elabora i frammenti estratti e formula una risposta precisa e discorsiva, basandosi sulle informazioni appena recuperate.

2.2 Chunking (Suddivisione del Testo)

Il chunking è il processo con cui i documenti originali vengono divisi in blocchi più piccoli. È una fase essenziale perché i modelli LLM non riescono a leggere documenti interi tutti in una volta, e perché ci serve isolare solo le informazioni che rispondono davvero alla domanda dell'utente senza sporcare la ricerca con informazioni non necessarie.

Token-based Chunking

È il metodo più semplice, in cui il testo viene tagliato contando rigidamente un numero fisso di parole o token. Spesso viene lasciata una piccola porzione di testo condivisa (chiamata "overlap") tra la fine di un blocco e l'inizio del successivo, in modo da non spezzare mai un discorso a metà. Usare blocchi molto piccoli è utile se bisogna estrarre risposte secche e fattuali con estrema precisione. Al contrario, usare blocchi più grandi è necessario se vogliamo che il modello comprenda un contesto più ampio e discorsivo.

L'overlap va scelto con cura, infatti se è troppo basso perdiamo il senso della frase, mentre se è troppo alto c'è il rischio di sprecare risorse per salvare nei database informazioni doppie.

Sentence-based Chunking

A differenza del metodo precedente, questo metodo rispetta la punteggiatura, come i punti alla fine delle frasi. In questo modo otteniamo blocchi di lunghezza variabile, ma che sono perfetti dal punto di vista grammaticale e logico. È un'ottima scelta quando abbiamo a che fare con documenti discorsivi, dove spezzare una frase a metà rovinerebbe il significato rendendo impossibile il recupero del testo.

Semantic Chunking

È il metodo più intelligente in assoluto, ma anche quello più computazionalmente esoso. Questa tecnica segmenta il testo basandosi sulle variazioni semantiche, infatti valuta il significato intrinseco delle frasi attraverso modelli di embedding e genera un nuovo blocco solo nel momento in cui rileva un effettivo cambio di argomento. È perfetto per i testi in cui gli argomenti sono molto misti e vari, perché crea dei chunk densi e compatti dal punto di vista semantico, anche se impiega un tempo molto maggiore per analizzare i documenti.

2.3 Modelli di Embedding

I modelli di embedding vengono utilizzati per mappare frammenti di testo in rappresentazioni vettoriali, strutturandoli in modo tale che frasi con un significato affine vengano posizionate vicine tra loro. Questa caratteristica fondamentale consente al sistema di eseguire ricerche basate su concetti e sfumature semantiche, superando i limiti della tradizionale ricerca per corrispondenza esatta delle singole parole.

In fase di progettazione è possibile optare per modelli più compatti, che producono vettori di dimensioni ridotte. Questi garantiscono un'elaborazione rapida e un minor consumo di risorse, risultando ideali per elaborazioni basilari. Al contrario, esistono modelli architetturalmente più pesanti e con una maggiore densità parametrica, capaci di catturare sfumature semantiche complesse e per questo adatti a domini fortemente tecnici. La scelta del modello ottimale si basa tipicamente su benchmark standardizzati (come la classifica MTEB), i quali valutano oggettivamente le prestazioni al fine di individuare il miglior compromesso computazionale tra velocità di inferenza e precisione semantica.

2.4 Query Transformation

Spesso le domande che facciamo ai chatbot sono scritte male, sono troppo brevi o non hanno contesto. Le tecniche di trasformazione della query servono proprio a correggere e arricchire la nostra domanda prima ancora di fare la ricerca nel database.

HyDE (Hypothetical Document Embeddings)

Questa tecnica chiede all'LLM di inventare subito una risposta teorica alla nostra domanda. Anche se questa risposta inventata potrebbe contenere errori, userà sicuramente parole molto vicine all'argomento. A questo punto, usiamo questa risposta finta per cercare nel database i documenti veri che le assomigliano. Funziona benissimo per collegare in modo intelligente una domanda corta a un documento tecnico molto lungo.

Multi-Query

Qui chiediamo all'LLM di prendere la nostra domanda e di scriverla in tanti modi diversi, usando anche sinonimi. Poi facciamo la ricerca per tutte queste varianti contemporaneamente e uniamo i risultati. È utile per non perdersi documenti importanti solo perché il prompt utilizza un verbo specifico, mentre il documento ne utilizza un altro.

RAG Fusion

È l'evoluzione diretta della Multi-Query. Dopo aver cercato con diverse domande alternative, unisce e riordina i risultati incrociando le classifiche tramite l'algoritmo Reciprocal Rank Fusion. In questo modo, i documenti che hanno risposto bene a più varianti della domanda finiscono automaticamente in cima, assicurandoci di recuperare solo le informazioni più pertinenti alla domanda fornita.

2.5 Metodi di Retrieval

Il retrieval è la fase di recupero, in cui il RAG decide quali blocchi di testo estrarre dal database per rispondere alla domanda fornita.

Vector Similarity

Questo approccio quantifica la pertinenza calcolando la vicinanza tra il vettore generato dalla query e i vettori dei chunk indicizzati. Il suo principale vantaggio risiede nella capacità di astrarre la ricerca dal vocabolario specifico, infatti il sistema riesce a individuare frammenti di testo concettualmente identici alla domanda dell'utente e gestire intrinsecamente i sinonimi, garantendo il recupero delle informazioni anche in totale assenza di corrispondenza lessicale esatta.

Keyword Extraction

Questo approccio si basa sul matching esatto delle parole chiave. Sebbene non sia in grado di cogliere sfumature semantiche o gestire nativamente i sinonimi, valuta la rilevanza calcolando la frequenza dei termini della query all'interno dei documenti, normalizzata per la lunghezza del testo. Risulta uno strumento indispensabile in scenari applicativi che richiedono il recupero di acronimi o codici alfanumerici, dove la corrispondenza lessicale esatta risulta prioritaria rispetto all'affinità concettuale.

Hybrid Search

La ricerca ibrida mira a unire i punti di forza della ricerca semantica e di quella lessicale. Il sistema interroga il database parallelamente su entrambi i fronti, ottenendo due graduatorie distinte dei documenti più rilevanti. Queste liste vengono successivamente normalizzate e fuse in un'unica classifica finale mediante l'algoritmo di aggregazione Reciprocal Rank Fusion. L'approccio ibrido garantisce un recupero delle informazioni altamente bilanciato e resiliente, mitigando le debolezze individuali dei singoli metodi.

Reranking Neurale

Indipendentemente dalla tecnica adottata nella fase di ricerca iniziale, è possibile incrementare l'accuratezza del sistema introducendo una fase successiva di Reranking. Tipicamente, il retrieval iniziale si basa su architetture *Bi-Encoder*, le quali elaborano la query e i documenti in modo separato per garantire un recupero estremamente rapido all'interno del database. Al contrario, il reranking sfrutta modelli neurali avanzati chiamati *Cross-Encoder*, i quali ricevono in input e processano congiuntamente sia la query dell'utente che ciascun frammento pre-selezionato. Questo confronto incrociato e simultaneo permette al modello di cogliere dipendenze logiche e semantiche molto più profonde rispetto al semplice confronto tra vettori separati eseguito dal Bi-Encoder. Pur aggiungendo un costo computazionale non trascurabile, questo passaggio filtra i falsi positivi e riordina i documenti con altissima precisione, assicurando che all'LLM vengano inoltrati esclusivamente i contesti della massima rilevanza.

2.6 LLM e Parametri di Generazione

Il Large Language Model rappresenta il modulo terminale dell'architettura RAG, incaricato della generazione testuale. Esso riceve in input il prompt dell'utente arricchito dal contesto recuperato durante la fase di retrieval, con il compito di sintetizzare una risposta coerente in linguaggio naturale.

Finestra di Contesto

La quantità di testo che l'LLM può elaborare in una singola richiesta è limitata dalla sua finestra di contesto. Fornire un numero eccessivo di documenti estratti peggiora le prestazioni del modello, che finisce per dimenticare le informazioni centrali (Lost in the Middle). Al contrario, un contesto troppo ristretto non fornisce dati sufficienti per formulare una risposta completa.

Parametri (Temperatura, Top-k, Top-p)

Il carattere e lo stile di risposta dell'LLM possono essere regolati attraverso alcuni parametri matematici. Il Top-k e il Top-p servono a limitare il vocabolario a disposizione del modello, impedendogli di selezionare parole troppo rare o insolite durante la generazione del testo.

Per il nostro progetto, si è scelto di mantenere questi due parametri sui valori predefiniti dei modelli utilizzati, andando a variare unicamente la Temperatura. Questo parametro regola il livello di creatività e la variabilità della risposta. Abbiamo abbassato la Temperatura a un valore prossimo allo zero, costringendo il modello a comportarsi in modo estremamente deterministico e analitico, in modo da assicurare risposte basate esclusivamente sui documenti forniti.

Capitolo 3

Struttura del Progetto

3.1 Struttura dei File

```
|-- assets
|-- results
|-- src
|   |-- generative_phase
|   |   |-- analysis
|   |   |   |-- judge.py
|   |   |   |-- metrics.py
|   |   |   |-- plots.py
|   |   |-- configs
|   |   |   |-- config.py
|   |   |   |-- experiment_configs.json
|   |   |-- runner
|   |   |   |-- experiment_runner.py
|   |   |   |-- retriever.py
|   |   |-- main.py
|   |-- optimization_phase
|   |   |-- analysis
|   |   |   |-- metrics.py
|   |   |   |-- plots.py
|   |   |-- blocks
|   |   |   |-- block1_chunking_embedding.py
|   |   |   |-- block2_query_transformation.py
|   |   |   |-- block3_retrieval_reranking.py
|   |   |-- configs
|   |   |   |-- config.py
|   |   |   |-- experiment_configs.json
|   |   |-- main.py
|   |-- main.py
|   |-- requirements.txt
```

Per garantire ordine e modularità, abbiamo suddiviso fisicamente il codice sorgente del progetto in cartelle dedicate alle diverse fasi operative. Analizzando la directory principale `src`, troviamo la seguente architettura:

- `optimization_phase/`: Questa cartella contiene il codice dedicato all'ottimizzazione dei parametri del RAG.
  - `blocks/`: Contiene gli script responsabili di eseguire le estrazioni nel database per ogni blocco sperimentale.
  - `configs/`: Contiene i file JSON che definiscono i parametri di tutte le configurazioni analizzate nel capitolo successivo.
  - `analysis/`: Raccoglie gli script dedicati all'analisi dei risultati e alla generazione dei grafici.
- `generative_phase/`: Questa cartella contiene il codice dedicato all'interazione con i LLM.
  - `runner/`: Contiene gli script utilizzati per interfacciarsi con Ollama, inviando il contesto testuale recuperato e raccogliendo la risposta generata dall'LLM.
  - `analysis/`: Contiene gli script per generare grafici e le metriche necessarie a valutare qualitativamente le risposte. Include un LLM-as-a-judge che valuta quanto le risposte dell'AI si avvicinino alla *ground truth* attesa.

3.2 Librerie e Framework Utilizzati

Per sviluppare il progetto ci siamo affidati a librerie e framework specifici:

- **LlamaIndex**: Framework software progettato per semplificare la ricerca e l'estrazione di informazioni dai documenti tramite interfacce conversazionali basate su LLM, specializzato per la costruzione di architetture RAG.
- **Ollama** e **Groq**: Abbiamo utilizzato le API del servizio Groq per sfruttare la loro bassa latenza di risposta durante i test di Query Transformation. Ollama, invece, ci ha permesso di eseguire in locale i modelli utilizzati per la fase generativa.
- **Sentence-Transformers** e **FlagEmbedding**: Librerie open-source utilizzate per le elaborazioni vettoriali del sistema. La prima si è occupata di calcolare gli embedding dei testi, mentre la seconda ha fornito i modelli Cross-Encoder necessari per eseguire il Reranking.
- **PyMuPDF**: Libreria open-source utilizzata per estrarre il testo in modo rapido e pulito dai documenti PDF originali dei due domini.
- **HuggingFace**: Libreria utilizzata per scaricare e utilizzare i modelli open-source localmente, in particolare i modelli di embedding e di reranking.

3.3 Scelte Architetturali e Hardware

A livello di codice e infrastruttura, abbiamo dovuto prendere delle decisioni per ottimizzare i tempi e gestire il carico di lavoro:

- **Blocco 1 (Chunking ed Embedding)**: Abbiamo utilizzato modelli in locale caricati tramite la libreria Huggingface. Abbiamo deciso di inserire nel codice la possibilità di scegliere dinamicamente se utilizzare il processore o l'accelerazione hardware CUDA, in modo da aumentare la velocità in caso fosse disponibile una GPU NVIDIA. Nello specifico, per queste operazioni abbiamo eseguito i calcoli su un PC dotato di CPU Intel i7 9700K, 16GB di RAM e una GPU Nvidia RTX 2070 Super con 8GB di VRAM.
- **Blocchi 2 e 3 (Query Transformation e Retrieval)**: Per queste fasi abbiamo utilizzato modelli provenienti sia da Huggingface sia tramite le API di Groq. Poiché in questi due blocchi non era strettamente necessaria l'accelerazione CUDA, abbiamo effettuato i test su una secondo PC, equipaggiato con CPU Ryzen 5700x, 32GB di RAM e una scheda video AMD RX 7600XT con 16GB di VRAM.
- **Fase Generativa**: La generazione delle risposte è stata eseguita interamente in locale sfruttando i modelli caricati su Ollama. Per questa fase abbiamo utilizzato lo stesso PC dei blocchi 2 e 3.

Capitolo 4

Workflow del Progetto

Per rendere il lavoro ordinato abbiamo deciso di dividere il progetto in due macro-fasi distinte: la Fase di Ottimizzazione e la Fase Generativa.

4.1 La Fase di Ottimizzazione

In questa fase ci siamo concentrati sulla ricerca dei migliori parametri di configurazione del sistema RAG. Abbiamo strutturato questa ottimizzazione dividendola in tre blocchi di sperimentazione:

Blocco 1: Chunking ed Embedding

In questo primo blocco abbiamo testato sette diverse configurazioni per trovare il miglior compromesso tra la tecnica di suddivisione del testo (chunking) e il modello vettoriale (embedding):

- **C1 - Micro-Chunk (Token)**: Usa un approccio basato sui token con chunk molto piccoli (200 token, overlap 32) abbinato al modello leggero *all-MiniLM-L6-v2*. È un ottimo punto di partenza per ottenere risposte oggettive e precise.
- **C2 - Micro-Chunk (Sentence)**: Mantiene lo stesso modello leggero, ma usa un chunking basato sulle frasi per isolare e valutare l'effetto del taglio logico rispetto al taglio numerico dei token.
- **C3 - Small-Chunk (Token)**: Aumenta le dimensioni (256 token) e passa a un modello più performante (*BAAI/bge-base-en-v1.5*), puntando a ottimizzare la comprensione su contesti più specifici.
- **C4 - Medium-Chunk (Token)**: È la configurazione standard bilanciata. Usa chunk ampi (500 token, overlap 50) e il modello *bge-base*.
- **C5 - Medium-Chunk Large-Model (Token)**: Stesse dimensioni della C4, ma utilizza il potente modello *BAAI/bge-large-en-v1.5*.
- **C6 - Medium-Chunk (Sentence)**: Configurata su 350 unità sintattiche abbinate al modello *bge-base*, serve a capire se il rispetto della punteggiatura batte il conteggio rigido dei token su testi lunghi.

- **C7 - Semantic Chunking**: La configurazione più avanzata. Sfrutta un algoritmo intelligente per dividere il testo in base al suo significato, abbinato al modello *bge-large*.

Dopo aver eseguito i test su tutte queste configurazioni, abbiamo confrontato i risultati e selezionato quella con le prestazioni migliori, prestando molta attenzione a bilanciare la precisione con i tempi di esecuzione per garantire un sistema rapido ed efficiente.

Blocco 2: Query Transformation

Nel secondo blocco abbiamo mantenuto fissa la configurazione vincente del Blocco 1, concentrandoci sulle tecniche per ottimizzare la domanda dell'utente. Per generare le trasformazioni delle query ci siamo affidati alle API di Groq impostando una Temperatura a 0.1, in modo da mantenere la coerenza e il rigore tecnico dei termini, pur concedendo quel minimo di flessibilità necessaria per generare varianti della query.

Abbiamo confrontato quattro approcci distinti: partendo da una semplice Baseline senza alcuna trasformazione (in cui la domanda viene processata esattamente come è stata scritta), abbiamo testato la tecnica HyDE, la Multi-Query e, infine, la RAG Fusion, un'evoluzione che unisce la ricerca su più varianti a un riordinamento intelligente di tutti i risultati trovati.

Come nel blocco precedente, al termine dei test abbiamo analizzato i risultati di queste varianti per decretare l'approccio più efficace, bilanciando l'accuratezza con la latenza aggiuntiva introdotta dalle chiamate alle API.

Blocco 3: Retrieval e Reranking

Nell'ultimo blocco di ottimizzazione abbiamo esplorato le strategie per estrarre e filtrare i frammenti:

- **Baseline Vector e BM25**: Due test base per confrontare la ricerca semantica pura (Vector) e la ricerca lessicale pura (BM25), estraendo direttamente i 5 migliori chunk.
- **Baseline Hybrid**: Unisce ricerca vettoriale e per parole chiave, estraendo 20 documenti per poi restituire i 5 migliori fondendoli con l'algoritmo RRF.
- **Reranking (k=10 e k=20)**: Sfrutta un cross-encoder per analizzare e riordinare attentamente un insieme allargato di documenti estraendone 10 nella versione leggera o 20 nella combinazione profonda, per poi scremarli e restituire unicamente i 5 frammenti più pertinenti.

Analogamente ai blocchi precedenti, al termine di questi test finali abbiamo valutato le metriche per individuare la strategia di estrazione definitiva, ricercando sempre il bilanciamento ottimale tra l'accuratezza del recupero e la latenza del sistema.

4.2 La Fase Generativa

Identificata la configurazione vincente per il RAG, siamo passati alla fase generativa testando 12 configurazioni diverse. Abbiamo incrociato l'utilizzo di quattro LLM eseguiti in locale tramite Ollama con tre diverse strategie di prompting.

Modelli Utilizzati

Per coprire un ampio spettro di capacità, abbiamo selezionato quattro LLM di diversa grandezza e architettura:

- *Llama 3.2 (3B)*: Il modello ultraleggero di Meta. Lo abbiamo scelto come base per valutare le prestazioni minime che un modello con risorse molto limitate può garantire.
- *Llama 3.1 (8B)*: Il modello standard di riferimento della famiglia Llama. Lo abbiamo inserito per fare un confronto diretto con la versione ultraleggera e capire se l'aumento di parametri porta a un miglioramento sensibile nella capacità di ragionamento.
- *Qwen 2.5 (7B)*: Un modello sviluppato da Alibaba, inserito per confrontarlo peso simile con l'equivalente Llama 8B e valutarne le differenze di prestazioni.
- *Gemma 2 (9B)*: Sviluppato da Google. Essendo il più pesante del gruppo, viene utilizzato da termine di paragone per testare le massime prestazioni teoriche ottenibili sulla nostra macchina locale.

Strategie di Prompting

Per ognuno dei 4 modelli abbiamo testato:

- **Standard Prompt**: L'istruzione classica in cui passiamo i documenti all'LLM e chiediamo la risposta diretta.
- **Step-Back Prompting**: Chiediamo all'LLM di estrapolare prima un concetto generale dalla domanda, e solo dopo di usarlo per cercare e fornire la risposta.
- **Chain of Thought**: Chiediamo esplicitamente all'LLM di ragionare passo per passo prima di dare la soluzione finale, per limitare ulteriormente le allucinazioni.

Abbiamo eseguito la pipeline tenendo una Temperatura ferma a 0.0 (per azzerare la creatività del modello ed evitare che potesse inventare informazioni) abbinata a un approccio *Zero-Shot* (ovvero senza fornire all'LLM alcun esempio di risposta precompilata nel prompt). In questo modo abbiamo valutato le prestazioni finali del nostro sistema su 12 configurazioni combinate.

4.3 Metriche di Valutazione

Per valutare le prestazioni ottenute dal sistema RAG con le configurazioni testate, e per valutare le risposte finali degli LLM, abbiamo implementato due tipologie di valutazione distinte.

4.3.1 Metriche della Fase di Ottimizzazione

- **DocRecall@k**: Misura il numero di documenti unici rilevanti recuperati tra i primi k risultati, diviso per il numero totale di documenti attesi nella *ground truth*.

- **PassageRecall@k**: A differenza del DocRecall, si concentra sui singoli frammenti di testo. Siccome il chunk tagliato in automatico dal sistema potrebbe non corrispondere esattamente al passaggio di riferimento annotato nei nostri test, non possiamo usare una ricerca esatta. Nel nostro codice abbiamo quindi integrato un *Cross-Encoder* per calcolare la somiglianza semantica tra il chunk ottenuto e la frase di risposta corretta attesa.

- **R-Precision**: Misura la precisione del sistema limitandosi ai primi R documenti recuperati, dove R è esattamente il numero totale di documenti rilevanti presenti nella *ground truth* per quella specifica domanda.

- **DocNDCG@k e PassageNDCG@k**: Il Normalized Discounted Cumulative Gain valuta dove il documento recuperato si trova nella classifica. Se il documento utile viene trovato in decima posizione, il punteggio cala drasticamente rispetto al trovarlo al primo posto. L'abbiamo calcolata e analizzata nel Blocco 3 per giudicare l'efficacia dell'algoritmo di Reranking.

4.3.2 Metriche della Fase Generativa

Nella fase generativa abbiamo implementato il pattern **LLM-as-a-Judge**, utilizzando un LLM locale (*Llama 3.1 (8B)*) configurato appositamente per analizzare la logica e l'accuratezza della risposta generata.

Tramite un prompt inserito nel codice, il giudice riceve in ingresso la domanda originaria, la risposta ideale (Reference Answer) e la risposta effettivamente prodotta dall'LLM (Generated Answer). A quel punto formula una giustificazione ragionata e le assegna un punteggio binario:

- **1 (Corretto / Parziale)**: La risposta contiene la verità fondamentale della reference, affrontando correttamente la domanda, anche se omette dettagli minori. Nel caso di domande *unanswerable*, il punteggio 1 viene assegnato se il modello riconosce correttamente l'assenza di informazioni nel contesto e si astiene dall'inventare una risposta.

- **0 (Errato / Allucinazione)**: La risposta è sbagliata, imprecisa, produce allucinazioni oppure non risponde affatto alla domanda posta. Nel caso di domande *unanswerable*, il punteggio 0 scatta qualora il modello decida di allucinare una risposta inventandola.

Questo meccanismo di LLM-as-a-Judge ci ha permesso di automatizzare i test qualitativi su centinaia di risposte generate, mantenendo sempre una soglia di giudizio uniforme.

Capitolo 5

Domini Analizzati

Per essere sicuri che il nostro RAG funzionasse bene in scenari reali, abbiamo deciso di testarlo esplorando due domini (basi di conoscenza) completamente diversi tra loro, in modo da capire se le prestazioni del sistema cambiassero in base al tipo e allo stile dei documenti forniti.

5.1 Composizione e Scelta dei Domini

Al fine di garantire la riproducibilità dei benchmark e motivare la scelta sperimentale, abbiamo definito rigorosamente due corpus documentali. La selezione mirava a testare il sistema RAG su due poli opposti dell'elaborazione del linguaggio naturale:

- **Dominio Tecnico (RFC)**: Il primo dominio è costituito da 39 documenti *Request for Comments*, estratti direttamente dal repository ufficiale dell'IETF. Il dataset ha un peso complessivo di circa 21,7 MB ed è caratterizzato da una quasi totale assenza di immagini. Si tratta di testi fortemente tecnici e prescrittivi che descrivono standard e protocolli di rete, aventi una struttura molto rigida, abbondanza di acronimi e sezioni schematiche. Abbiamo scelto questi documenti per testare la capacità del RAG di comportarsi come un estrattore di precisione in ambiti molto specifici.
- **Dominio Umanistico (WW1)**: Il secondo dominio è composto da testi storiografici sulla Prima Guerra Mondiale, selezionati da archivi accademici open-source e librerie di pubblico dominio. A differenza del primo dominio, questo set contiene solo 6 documenti, i quali però risultano molto più estesi e ricchi di immagini, raggiungendo un peso complessivo di 22,2 MB. La scrittura è letteraria, discorsiva, ricca di sinonimi, descrizioni lunghe, riferimenti incrociati a date e nomi di persone o luoghi. Lo scopo era valutare la capacità del sistema di comprendere il contesto umano e collegare argomenti diluiti su paragrafi o pagine diverse.

5.2 Struttura delle Domande

Per testare il sistema, abbiamo preparato dei file JSON contenenti un lungo elenco di domande. Abbiamo deciso di dividere le domande in due file separati: uno con 90 domande dedicato alla fase di ottimizzazione, e un altro con 150 domande per la fase di generazione. Analizzando la struttura che abbiamo deciso di utilizzare, ogni domanda possiede questi campi fondamentali:

- **id** e **question**: il codice identificativo della riga e il testo della domanda.
- **type**: indica se l'informazione per rispondere si trova in un solo documento (`single_doc`) oppure se richiede di unire informazioni provenienti da più file (`multi_doc`).
- **relevant_documents** e **reference_passages**: Questi campi contengono rispettivamente il nome esatto del documento e l'esatta frase del testo originale dove risiede la soluzione.
- **answer**: La risposta perfetta che vorremmo ricevere alla fine.

Capitolo 6

Analisi dei Risultati

In questo capitolo andremo ad analizzare i risultati ottenuti eseguendo i nostri test sui due domini scelti. Seguiremo passo dopo passo il flusso che abbiamo spiegato nel Capitolo 4, partendo dall'ottimizzazione dei parametri del RAG fino ad arrivare alla valutazione delle risposte generate dai LLM.

6.1 Ottimizzazione: Blocco 1 (Chunking ed Embedding)

Per iniziare, abbiamo eseguito le 7 configurazioni del primo blocco su entrambe le basi di conoscenza, per capire quale fosse la migliore strategia. Abbiamo misurato le prestazioni utilizzando le metriche di *DocRecall*, *PassageRecall* e *R-Precision*. Di seguito riportiamo i grafici delle metriche ottenute, calcolate sull'intero set di domande per i due domini.

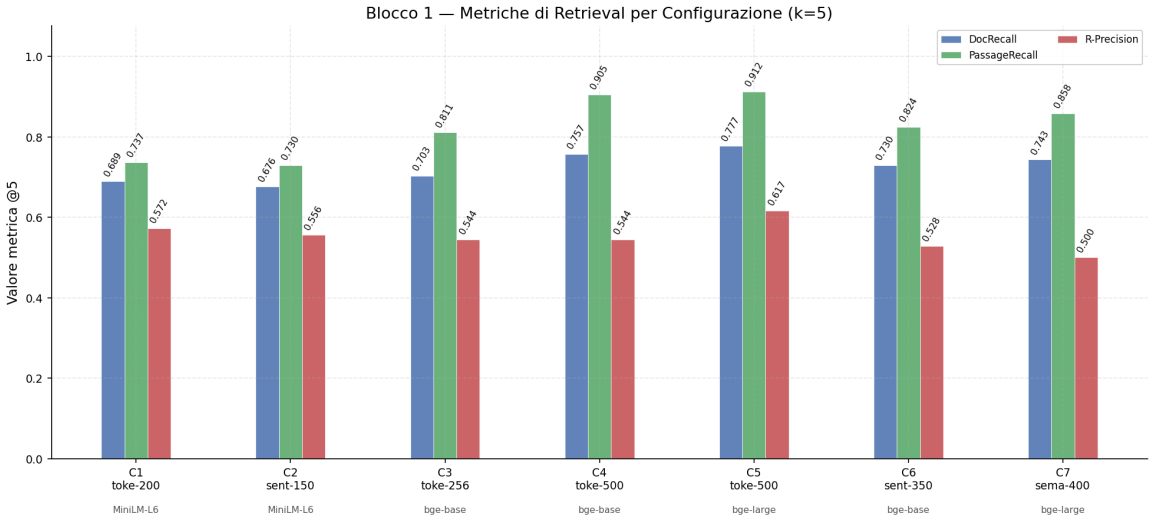


Figura 6.1: Metriche del Blocco 1 - Dominio RFC

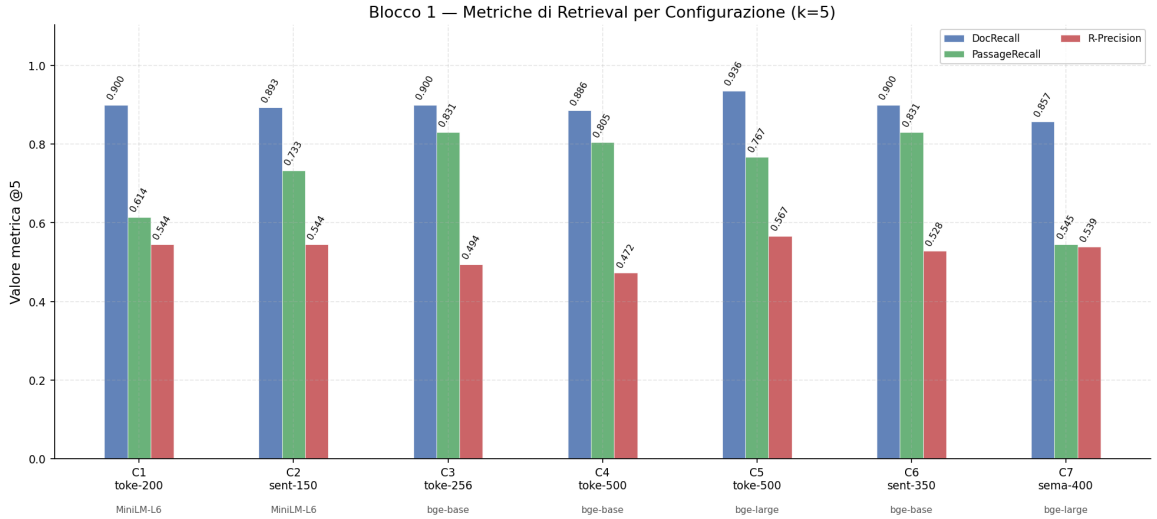


Figura 6.2: Metriche del Blocco 1 - Dominio WW1

Come è possibile notare dai grafici, le prestazioni misurate sono molto simili in entrambi i domini, pur con punteggi mediamente più alti per WW1 grazie al suo linguaggio discorsivo e più semplice, rispetto a quello denso e tecnico delle RFC.

Analizzando i grafici, le configurazioni C1 e C2 (micro-chunk) offrono un'elaborazione veloce ma faticano a catturare abbastanza contesto, mentre la C7 (taglio semantico) si è rivelata deludente, finendo per frammentare eccessivamente i testi narrativi. I punteggi iniziano a salire in modo significativo aumentando la dimensione della finestra di lettura (C3 e C4).

Tuttavia, la vincitrice è risultata la configurazione C5 (Medium-Chunk a 500 token con modello vettoriale *bge-large*), raggiungendo un eccellente 91% di PassageRecall su RFC e quasi il 94% di DocRecall su WW1. Si è dimostrata la scelta migliore perché i vettori del modello "large", abbinati a chunk abbastanza lunghi da contenere concetti interi senza tagliarli, garantiscono una comprensione semantica nettamente superiore rispetto ai modelli più leggeri.

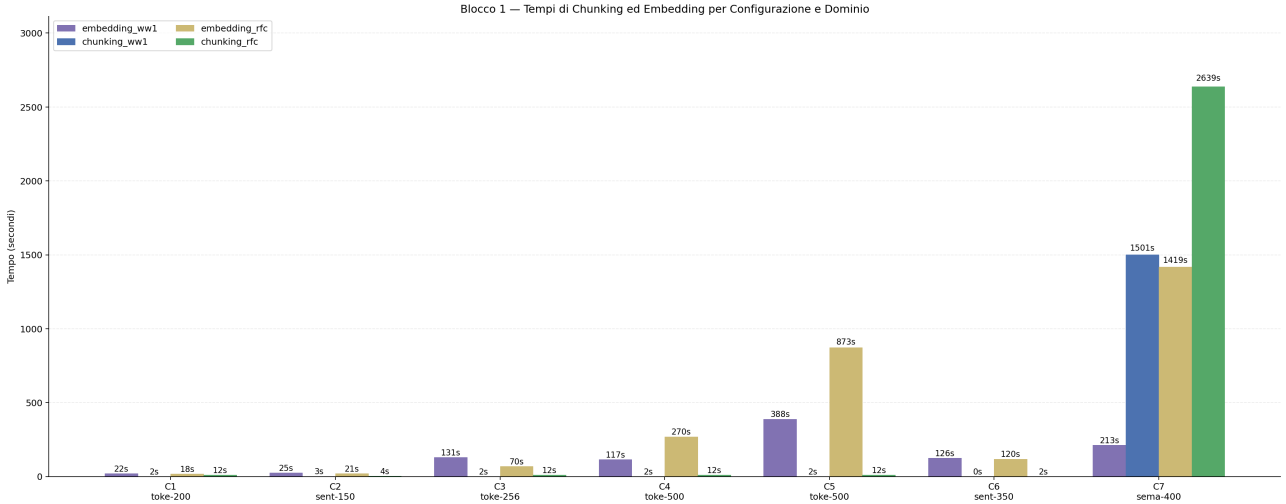


Figura 6.3: Tempi di elaborazione

Dai grafici emerge che la configurazione C5 richiede tempi di embedding superiori rispetto alle configurazioni più semplici a causa del peso maggiore del modello *bge-large*. E' stata comunque scelta come vincitrice poiché l'indicizzazione è un'operazione eseguita una tantum, dunque un tempo di caricamento maggiore è un compromesso accettabile per massimizzare la precisione, dato che la latenza di retrieval per l'utente rimane pressoché invariata.

6.2 Ottimizzazione: Blocco 2 (Query Transformation)

Passando al secondo blocco, abbiamo valutato l'impatto di diverse tecniche di Query Transformation. Per isolare il contributo di queste tecniche, tutti i test di questo blocco sono stati eseguiti utilizzando la configurazione migliore del blocco precedente (la Configurazione C5) e una baseline standard per il retrieval. Le quattro strategie testate sono state: none (assenza di trasformazioni), HyDE, Multi-Query e RAG Fusion. Di seguito riportiamo i grafici riguardanti le metriche ottenute per entrambi i domini.

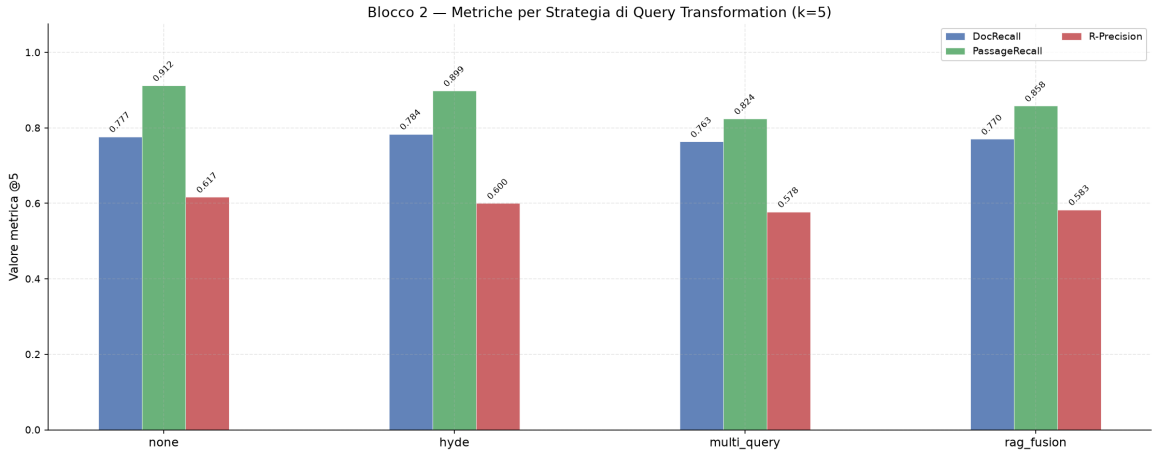


Figura 6.4: Metriche del Blocco 2 - Dominio RFC

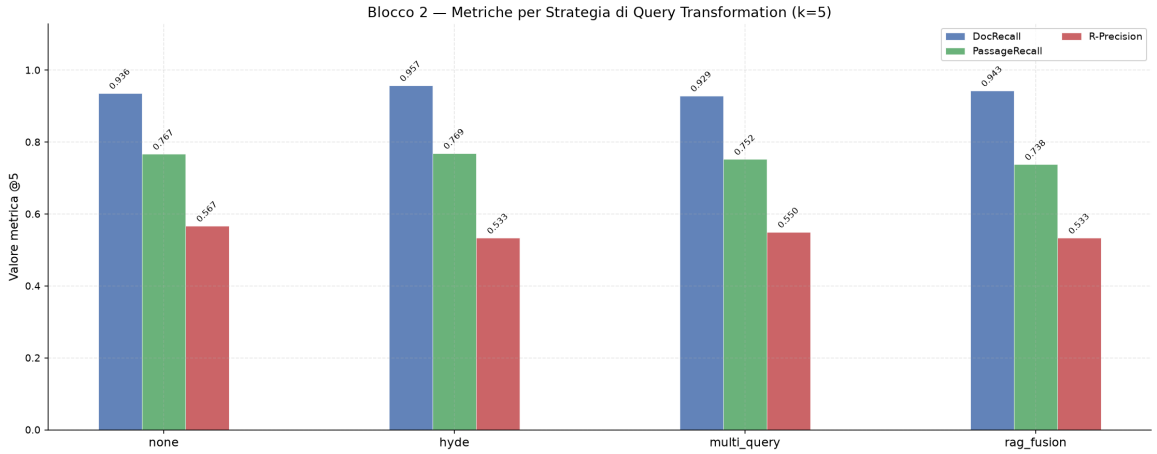


Figura 6.5: Metriche del Blocco 2 - Dominio WW1

Anche in questo caso, i due domini hanno ottenuto delle prestazioni molto simili. Come si nota dai grafici, la baseline ha eguagliato e in certi casi superato le tecniche più avanzate. La motivazione risiede nel fatto che le nostre query di test erano già chiare e ben formulate, dunque chiedere a un LLM di generare documenti ipotetici o di riscrivere più volte una query già precisa ha finito per introdurre del "rumore".

Oltre all'accuratezza, abbiamo esaminato anche la latenza media e il numero di token consumati dalle chiamate all'LLM necessarie per trasformare le query.

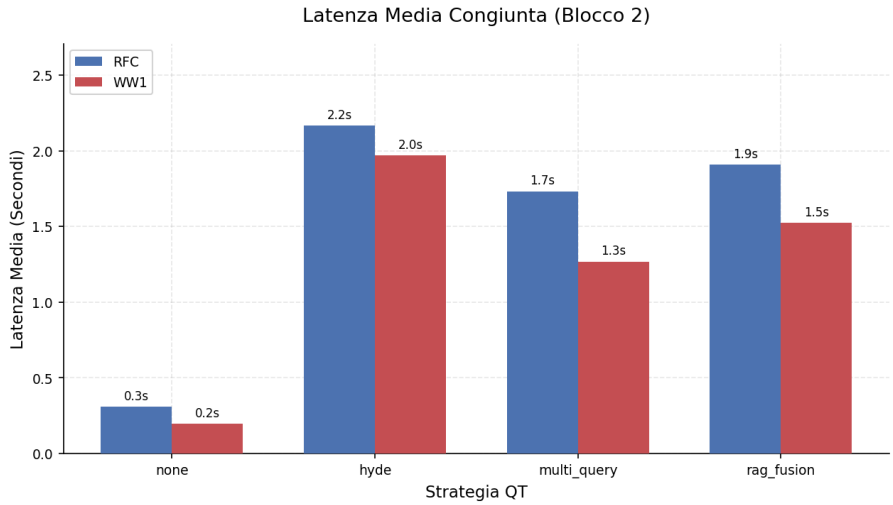


Figura 6.6: Latenza media

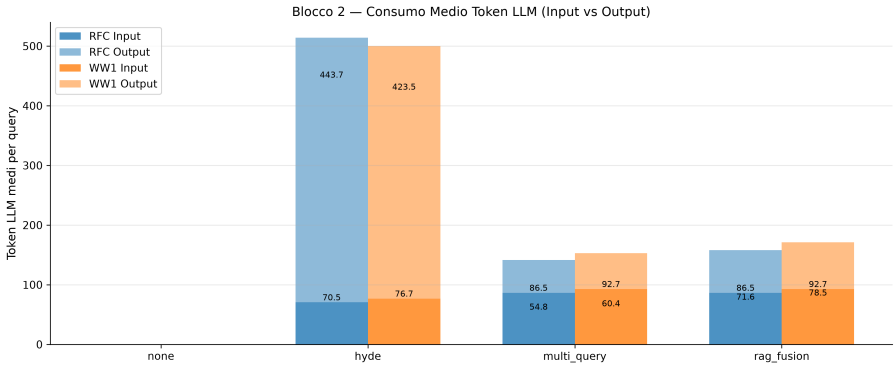


Figura 6.7: Token utilizzati

Confermiamo la scelta della configurazione base come vincitrice di questa fase. Essa ha ottenuto prestazioni praticamente pari o superiori a quelle delle architetture più complesse, garantendo però al contempo un utilizzo nullo di token e una latenza nettamente minore, rivelandosi la scelta più logica ed efficiente.

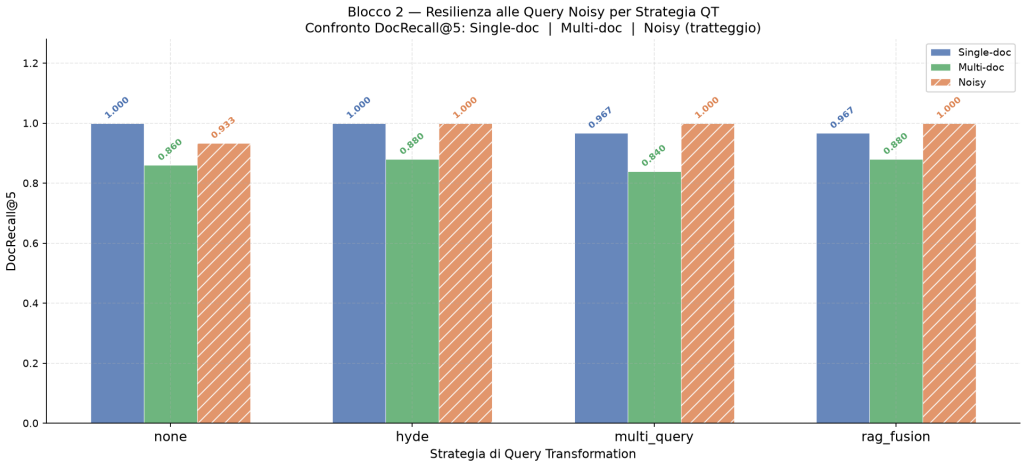


Figura 6.8: Impatto delle query "noisy" - Dominio WW1

Durante i test di questo blocco abbiamo valutato anche l'elasticità del sistema sottoponendogli un piccolo set di domande "noisy" (scritte in modo colloquiale, con slang o punteggiatura eccessiva). Le prestazioni su queste query si sono rivelate molto buone, toccando spesso punteggi massimi. Questo fenomeno è dovuto alla natura dei modelli di embedding, che imparano a ignorare completamente il rumore, concentrando il calcolo semantico solo sulle vere parole chiave della frase. In aggiunta, trattandosi di un campione di domande numericamente ridotto rispetto a quello standard, è stato statisticamente più facile raggiungere in certi casi il massimo delle prestazioni.

6.3 Ottimizzazione: Blocco 3 (Retrieval e Reranking)

Per completare la fase di ottimizzazione, abbiamo testato le 7 configurazioni relative ai metodi di estrazione finali, mantenendo fissi i parametri ottimali individuati nei due blocchi precedenti. Di seguito riportiamo i grafici delle metriche di recupero per i due domini.

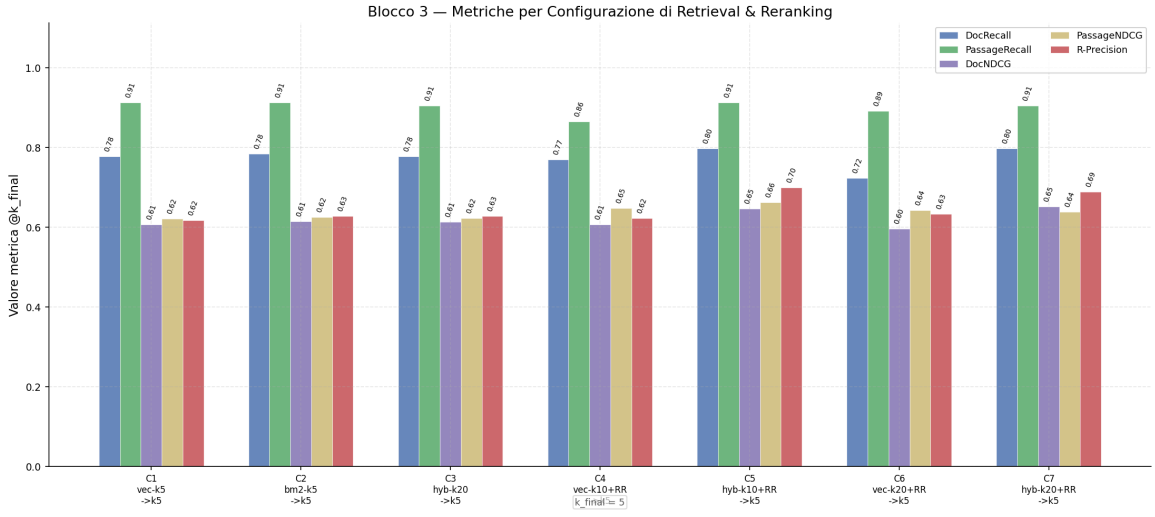


Figura 6.9: Metriche del Blocco 3 - Dominio RFC

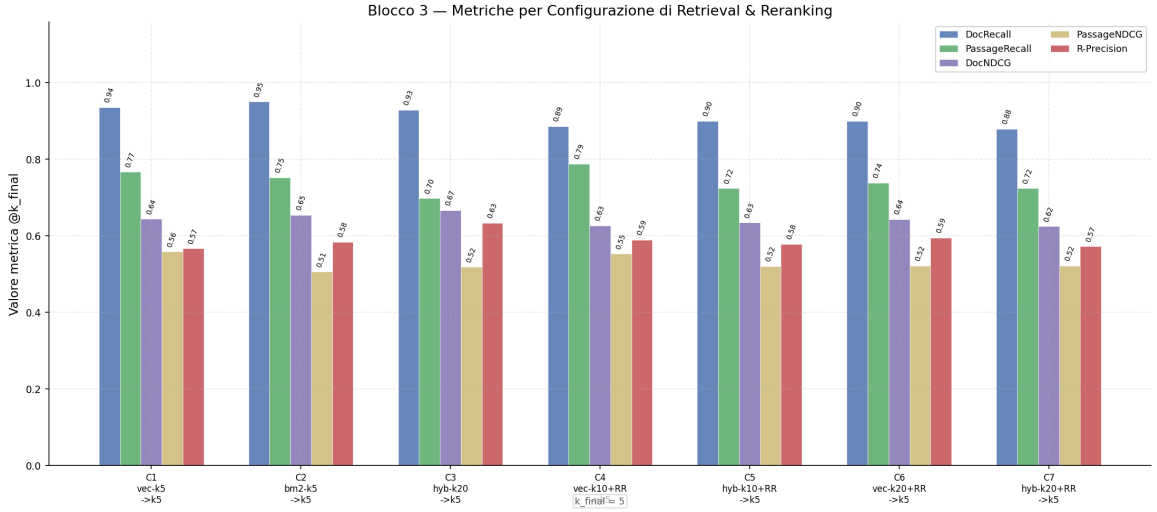


Figura 6.10: Metriche del Blocco 3 - Dominio WW1

Osservando le metriche possiamo notare che l'adozione di tecniche complesse non ha garantito un reale incremento nelle prestazioni. Al contrario, la configurazione basata sulla pura ricerca lessicale (BM25) ha registrato prestazioni ottime in entrambi i domini.

Nel dominio RFC, fortemente tecnico e ricco di acronimi o codici standard, la ricerca per corrispondenza esatta dei termini (BM25) risulta imbattibile. Nel dominio narrativo WW1, sebbene ci si aspettasse un vantaggio della ricerca semantica, le domande di test richiedevano molto spesso il recupero di entità specifiche (nomi di generali, date precise o luoghi). L'elevata unicità di queste keyword ha permesso al BM25 di individuare i paragrafi corretti senza il bisogno di un'astrazione semantica, rendendo superflua una *Hybrid Search*.

Di seguito analizziamo i tempi di risposta richiesti al sistema per recuperare ed eventualmente riordinare i chunk estratti.

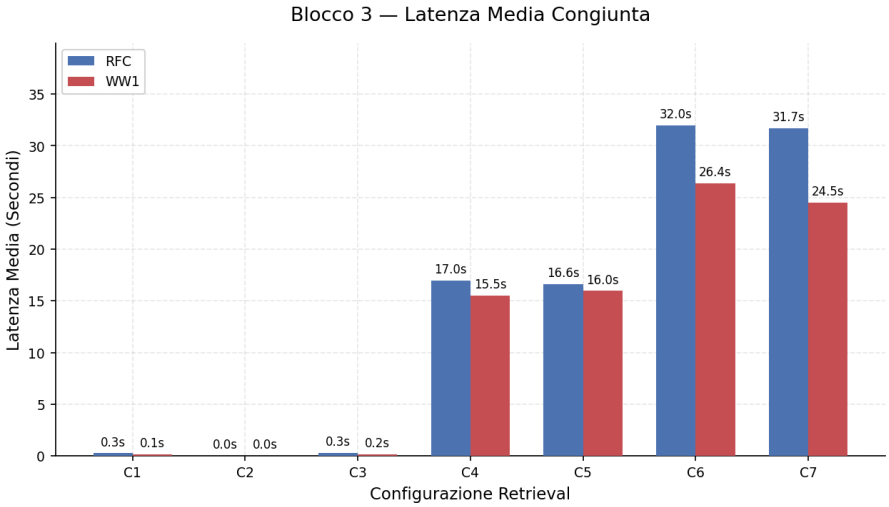


Figura 6.11: Latenza media di estrazione

Come prevedibile, l'introduzione di un Cross-Encoder causa un collo di bottiglia computazionale che fa impennare drasticamente i tempi di latenza. Avendo dimostrato che la ricerca lessicale

BM25 riesce a eguagliare le architetture ibride grazie all'alta specificità delle nostre domande, il costo computazionale aggiuntivo del reranking risulta ingiustificato nel nostro scenario. Per questo motivo, abbiamo confermato il BM25 puro come strategia definitiva, massimizzando l'accuratezza con una latenza pressoché nulla.

6.4 Fase Generativa

Completata la fase di ottimizzazione e individuati i parametri di retrieval ideali (chunk size di 500, embedding bge-large-en-v1.5, ricerca bm25 senza reranking, nessun query transformation), siamo passati all'esecuzione della fase generativa. In questa fase abbiamo testato le 12 configurazioni descritte nei capitoli precedenti, valutando qualitativamente la capacità dei vari modelli LLM di formulare risposte adeguate. Di seguito riportiamo i grafici delle metriche ottenute per i due domini.

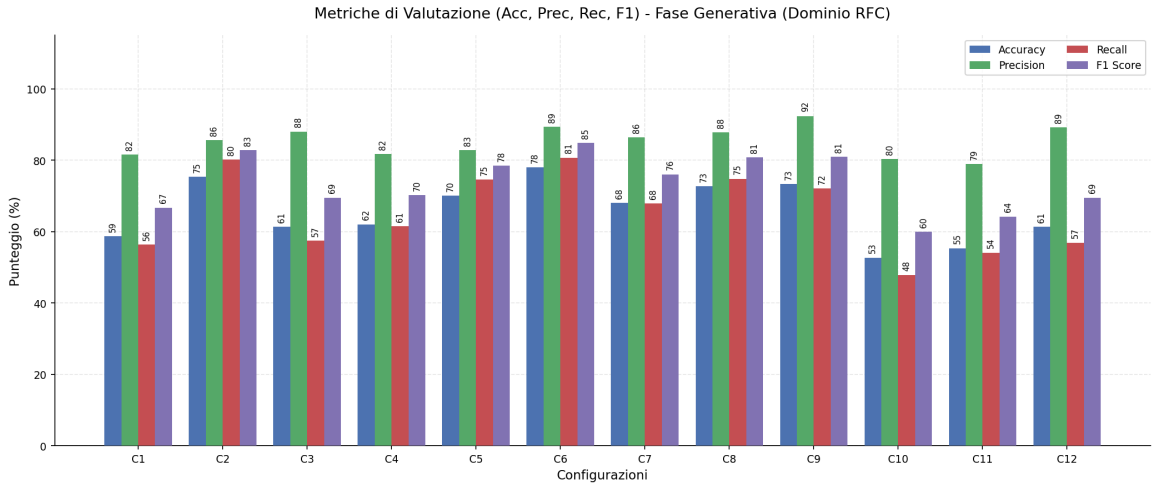


Figura 6.12: Metriche della Fase Generativa - Dominio RFC

Analizzando i risultati sul dominio tecnico RFC, spiccano come vincitori le configurazioni C6 e C2. Entrambe utilizzano modelli della famiglia Llama 3, supportate da tecniche di prompting avanzate (Chain of Thought e Step-Back). Osservando i benchmark pubblici come MATH o GSM-8K, la famiglia Llama 3 ha dimostrato un'ottima predisposizione analitica e logico-matematica. Per questo motivo, guidati da un prompt che li forza a ragionare step-by-step, sono risultati i migliori per decodificare e sintetizzare regole di rete o standard tecnici complessi. Al contrario, i modelli della famiglia Gemma 2 non sono riusciti ad adattarsi alla rigidità del dominio, registrando i punteggi più bassi.

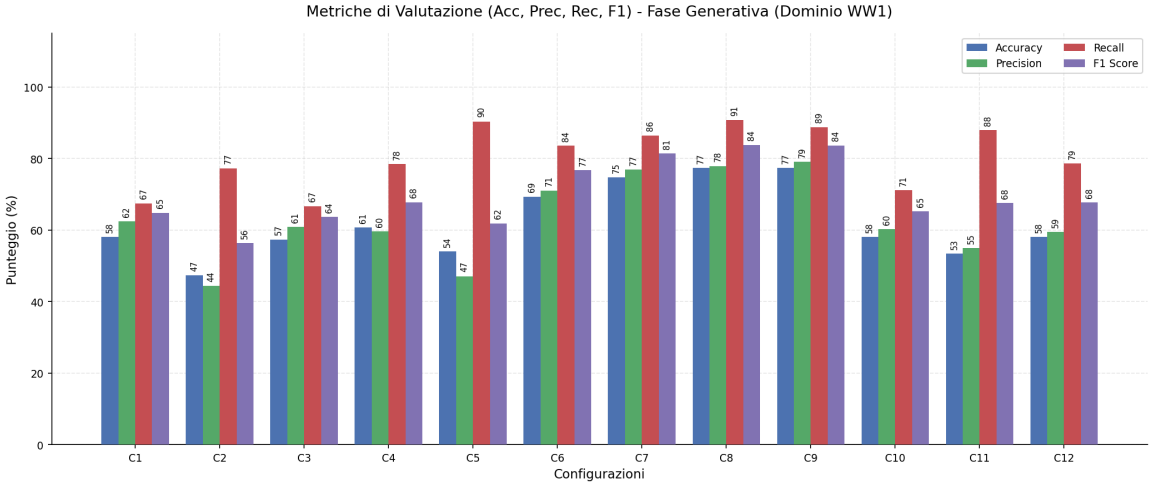


Figura 6.13: Metriche della Fase Generativa - Dominio WW1

Nel dominio narrativo e storico WW1, lo scenario cambia a favore delle configurazioni C7, C8 e C9. Queste configurazioni impiegano tutte il modello Qwen 2.5 7B, il quale domina a prescindere dalla strategia di prompt applicata. Le prestazioni registrate sono imputabili alle sue eccellenti doti di *reading comprehension* dimostrate dagli ottimi risultati ottenuti su benchmark internazionali come MMLU e IFEval. Questo gli permette di analizzare e rielaborare frammenti testuali descrittivi senza allucinazioni, surclassando non solo i modelli più compatti come Llama 3.2 3B, ma dimostrandosi nettamente superiore anche a modelli di taglia maggiore come Gemma 2 9B.

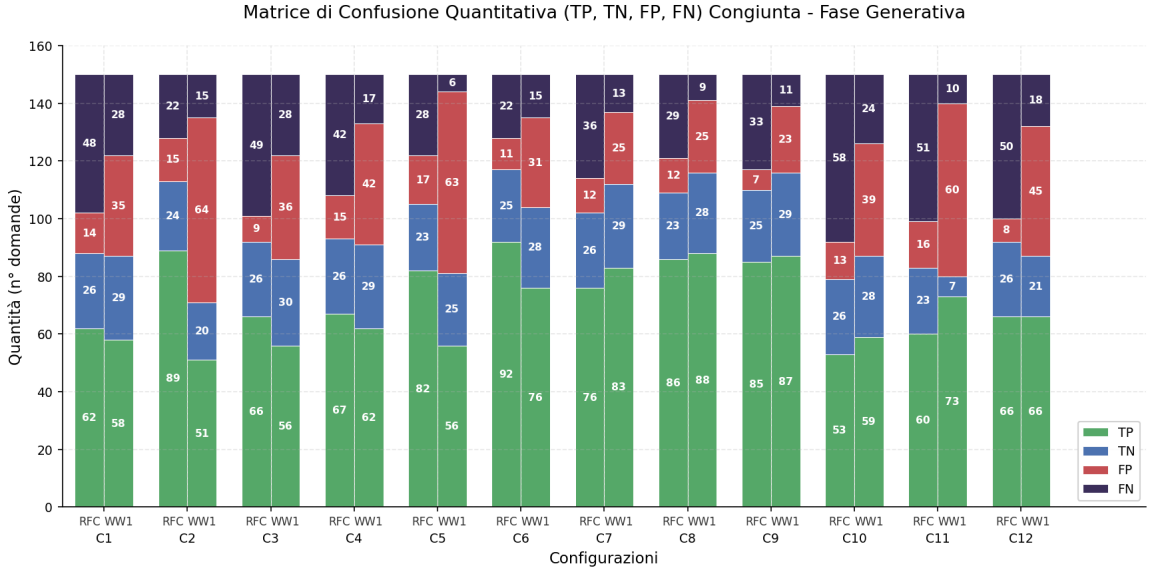


Figura 6.14: Matrice di Confusione

Osservando la matrice di confusione è possibile notare che nel dominio WW1 c'è un numero di falsi positivi decisamente più alto rispetto al dominio RFC. Questo problema è da attribuire al modello usato come giudice. In particolare, nel dominio WW1 i modelli tendono a fornire risposte molto lunghe e descrittive, ed il nostro giudice ha spesso valutato come errate queste

risposte perché contenevano dettagli aggiuntivi storicamente corretti ed estratti dal contesto, ma che non erano presenti nella risposta di riferimento. Nel dominio RFC, invece, le risposte sono molto più schematiche e dirette, quindi il giudice è riuscito a valutarle in modo preciso senza andare in confusione. Di conseguenza, possiamo dire che i punteggi ottenuti nel dominio WW1 non sono pienamente attendibili a causa dei limiti del modello usato per la valutazione.

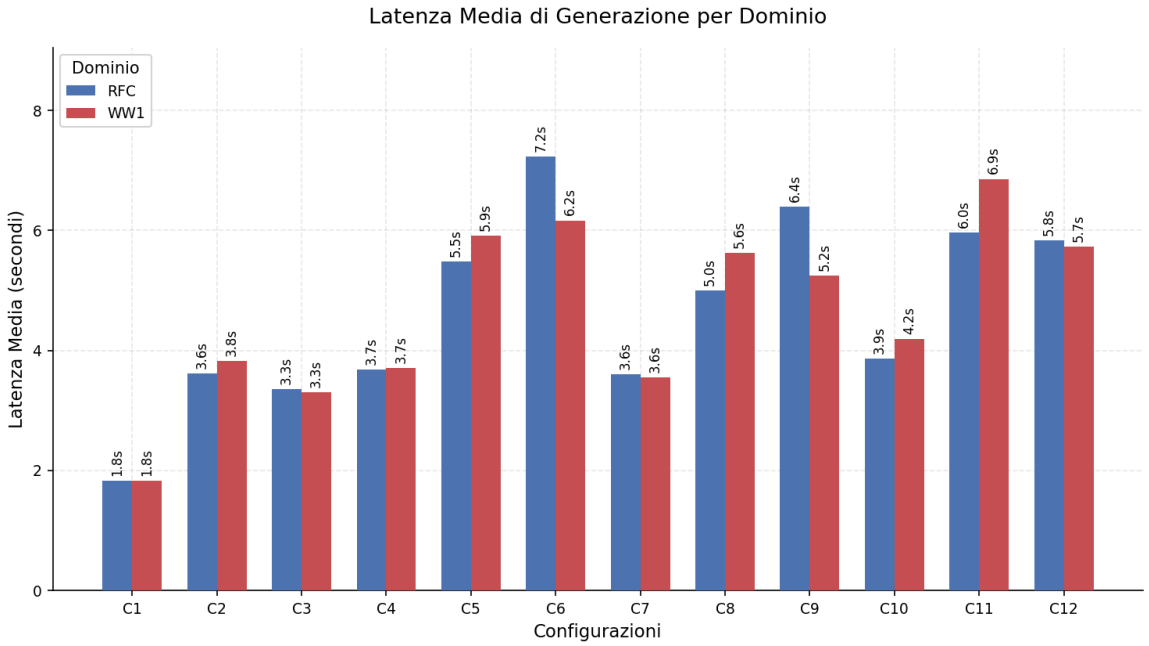


Figura 6.15: Latenza media di generazione per configurazione

Oltre alle metriche qualitative, abbiamo valutato le configurazioni anche sotto il profilo dei tempi di risposta. Considerando la latenza media, i vincitori assoluti risultano essere C2 per il dominio RFC e C7 per il dominio WW1. La configurazione C2, pur offrendo performance paragonabili a C6, utilizza un modello notevolmente più compatto, abbattendo di fatto i tempi di inferenza. Per WW1, C7 ottiene punteggi eccellenti pur avvalendosi di un prompt Standard, riuscendo a generare la risposta finale in tempi decisamente più rapidi rispetto alle varianti C8 e C9, confermandosi come il compromesso ideale tra prestazioni ed efficienza.

Analisi degli Errori Residui

Nonostante le configurazioni vincitrici abbiano registrato prestazioni eccellenti, l'analisi delle risposte errate rivela alcune lacune persistenti. Nel dominio RFC il modello Llama 3.2 3B presenta ancora dei falsi negativi, concentrati quasi esclusivamente sulle domande multi-documento. Infatti, l'LLM di dimensione ridotta fatica ad aggregare vincoli sparsi su più chunk, finendo spesso per dimenticare o omettere le regole secondarie.

Nel dominio WW1, il modello Qwen 2.5 7B mostra una tendenza opposta, producendo alcuni falsi positivi. Interrogato con domande *unanswerable*, il modello tende a farsi trascinare dalle proprie nozioni pre-addestrate sulla guerra, finendo per allucinare date o eventi storicamente veri, ma di fatto del tutto assenti nei documenti forniti al sistema.

Capitolo 7

Conclusioni

In questo progetto abbiamo sviluppato e ottimizzato un sistema RAG testandolo su due domini contrapposti: tecnico (RFC) e umanistico (WW1). L'obiettivo era dimostrare come la personalizzazione dei parametri possa portare ad un aumento delle prestazioni, con l'obiettivo di eliminare le allucinazioni e garantire risposte precise.

Nella fase di **Ottimizzazione** abbiamo individuato la configurazione ideale del RAG: un approccio basato su chunk da 500 token, embedding *bge-large-en-v1.5* e ricerca puramente lessicale (BM25). Questa configurazione, priva di algoritmi di reranking, ha garantito il miglior bilanciamento assoluto tra le metriche e tempi di latenza ottenuti in entrambi i domini.

Nella **Fase Generativa**, l'analisi mostra l'assenza di un modello ottimale per ogni scenario. Nel dominio tecnico (RFC), la configurazione basata su *Llama 3.2 3B* con *Step-Back Prompting* si è dimostrata la soluzione ideale, garantendo massima precisione e tempi di inferenza ridotti. Nel dominio umanistico (WW1) invece, *Qwen 2.5 7B* abbinato a un *Standard Prompt* ha superato anche modelli più grandi, dimostrando che la sua alta comprensione del testo è il fattore vincente per processare documenti narrativi e descrittivi.

Limiti e Sviluppi Futuri

Sebbene il sistema abbia dimostrato ottime prestazioni, è fondamentale evidenziare alcune limitazioni architetturali:

- **Dati Multimodali e Tabelle**: Il RAG attuale elabora solo testo puro. È cieco di fronte a immagini o grafici e fatica con le tabelle strutturate.
- **Staticità del Vector Store**: L'indicizzazione attuale richiede una rigenerazione completa del database per ogni nuovo documento aggiunto. In scenari reali servirebbero pipeline dinamiche per l'aggiornamento incrementale.
- **Natura "Black-Box"**: Le prestazioni finali restano comunque vincolate ai limiti cognitivi dell'LLM, a prescindere dall'accuratezza del retrieval.

In conclusione, i risultati ottenuti in questo studio dimostrano che l'efficienza di un sistema RAG dipende anche dal contesto in cui opera. Per minimizzare le allucinazioni e massimizzare le prestazioni generali, è indispensabile calibrare sia le strategie di retrieval che la scelta dei LLM generativi in base alle specifiche caratteristiche semantiche e strutturali dei dati a disposizione.